MODULE 06 · DOMAIN 2 - DATA PREPARATION

Choosing the right sources

Which data the application needs, where it should live, and what to keep out of the index.

SECTION 2 of 6 EXAM WEIGHT 14% MODULE 06 of 40

admin@sthithapragna.com

The model cannot quote a page you never gave it. **And it will happily quote a bad one.**

admin@sthithapragna.com

Five kinds of source, five different homes

THE KNOWLEDGE IS	EXAMPLE	IT BELONGS
General prose, same for all	Returns policy, manual	Retrieval index
A value about one entity	Delivery date of one order	Table, looked up by key
Live, changing by the second	Stock, seat availability	Tool on the live system
Aggregates over many rows	Sales by region	Tables, via SQL or Genie
Only in people's heads	Unwritten escalation rules	Nowhere yet: write it down

ON THE EXAM

When one question type fails, ask which row its answer lives in, and whether the app can reach it.

Look it up by key, or retrieve by meaning?

LOOK UP BY KEY WHEN

The answer is about one entity

- The user supplies an ID
- The value differs per customer
- It must be exact
- It lives in a table already

RETRIEVE BY MEANING WHEN

The answer is general knowledge

- Same answer for everyone
- It is written as prose
- Wording varies between questions
- It lives in documents

THE TIEBREAKER

If the scenario mentions an ID the user provides, the fix is almost never a document. It is a keyed lookup.

Where each kind of source lives

SOURCE	DATABRICKS HOME
Documents	Files in a Unity Catalog volume, parsed, then indexed in AI Search
Per-entity values, low latency	Feature Serving from an online feature store, backed by Lakebase
Tables and aggregates	Unity Catalog tables, via Genie Agents or UC function tools
Live external systems	Tools and MCP servers
Everything above	Governed in Unity Catalog, with lineage and permissions

ON THE EXAM

Databricks documents Feature Serving specifically for serving structured data to rag applications.

A good source is authoritative, current and complete

- **Authoritative:** the official version, not a copy in someone's email.
- **Current:** superseded versions removed, or clearly marked and filtered.
- **Complete:** covers the questions users will actually ask.
- **Permitted:** licensed and allowed for this use. Module 30 covers this.
- More documents is not better. Every irrelevant page competes with the right one.

> ON THE EXAM
>
> Quality of sources beats quantity. A smaller, clean corpus retrieves better than a larger, noisy one.

Work backwards from the questions

01
Collect
Real questions from users or support logs.

02
Map
Which source answers each one?

03
Find gaps
Questions with no source at all.

04
Fill
Add the source, or a tool, or write it down.

05
Keep
The questions become an evaluation set.

ON THE EXAM

A question type with no mapped source is a guaranteed failure. You can find it before launch, on paper.

Extraneous content that degrades retrieval

- Web furniture: navigation menus, cookie banners, footers, related-story links.
- Advertising and promotional copy mixed into real content.
- Off-topic material: sport in a technology corpus, other companies' news.
- Duplicates and near-duplicates, which crowd out variety in the top results.
- Superseded versions that contradict the current one.

ON THE EXAM

Noise costs twice. It wastes space in the prompt, and it can outrank the chunk you actually needed.

How to keep noise out

TECHNIQUE	REMOVES
Parse with structure, drop boilerplate	Menus, headers, footers, banners
Relevance classification before indexing	Off-topic articles and adverts, using ai_classify or an LLM
Deduplication by hash or similarity	Repeated and near-repeated passages
Version control of sources	Superseded documents
Metadata tags and query-time filters	Content relevant only to some users or dates

ON THE EXAM

Remove what nobody should ever retrieve. Tag what only some queries should retrieve.

Remove before indexing, or filter at query time?

REMOVE AT INGESTION WHEN

Nobody should ever see it

- Adverts and page furniture
- Off-topic material
- Duplicates
- Superseded versions, if never needed

TAG AND FILTER AT QUERY WHEN

Only some queries need it

- Different regions or products
- Date ranges
- Different user permissions
- Old versions kept for audit

THE TIEBREAKER

Asking the model to ignore bad chunks in the prompt is the weakest fix. It is still paying for them, and still reading them.

Harbourline's source inventory

01
Index
Current policy wordings, one version each.

02
Tag
Product, policy year, customer segment.

03
Look up
Customer's tier and claims, by policy number.

04
Exclude
Brochures, intranet news, old versions.

05
Capture
Escalation rules, written down first.

ON THE EXAM

The inventory is a design decision. It is where most rag quality is won or lost, before any model is involved.

An energy supplier's assistant answers tariff and policy questions well. Customers sign in, so the assistant knows each customer's account number.

It fails on questions like: what will my next bill be? Projected bills are calculated nightly and stored per account.

What change would most improve these answers?

A Add the billing policy documents to the retrieval index as well

B Fine-tune the model on a large sample of past customer bills

C Tell the model to estimate from the customer's typical usage

D Look up the projected bill by account number from its table

◆ PAUSE AND DECIDE

What change would most improve these answers?

A Billing policy documents

Explains how bills work. Contains nobody's actual bill.

B Fine-tune on past bills

Learns what bills look like, not this customer's next one.

C Estimate from usage

Invents a number when the real one already exists.

D Keyed lookup by account

The exact value exists, per account. Fetch it.

THE SIGNAL

Knows each customer's account number, and bills are stored per account. An ID plus a per-entity value means a keyed lookup.

A rag application answers questions about technology news. Its sources are scraped web pages containing navigation menus, cookie banners, adverts and related-story links, many about sport.

Retrieved chunks are often boilerplate or sport stories, and answers suffer.

What is the most effective fix?

A Strip boilerplate when parsing, and filter off-topic articles out

B Retrieve more chunks per query so good ones are more likely to appear

C Use a model with a larger context window so more pages fit at once

D Instruct the model to ignore adverts and sport stories in its prompt

◆ PAUSE AND DECIDE

What is the most effective fix?

A Clean and filter at ingestion

Removes noise before it can compete for retrieval.

B Retrieve more chunks

More noise comes back with the signal, and cost rises.

C Larger context window

Room for more noise is not less noise.

D Prompt: ignore adverts

Weakest fix. The noise still displaced the right chunks.

THE SIGNAL

Retrieved chunks are often boilerplate. The problem is in the index, so fix the index.

> An internal assistant indexes an HR folder that holds versions one, two and three of the expenses policy. The rules changed significantly in version three.
>
> Employees report answers that mix old and new limits in the same reply.

What should the engineer do?

A. Add a re-ranker to the retrieval step

B. Lower the temperature so answers are more consistent

C. Index only the current version, or tag versions and filter to the latest

D. Keep all versions and tell the model to prefer the newest one

◆ PAUSE AND DECIDE

What should the engineer do?

A **Re-ranker**

Ranks by relevance. Old and new versions are equally relevant.

B **Lower temperature**

Consistently mixed answers are still wrong.

C **Current version only, or filter**

Stops the contradiction from being retrieved at all.

D **Keep all, prefer newest**

Relies on the model spotting versions inside chunks. Fragile.

THE SIGNAL

Versions one, two and three are all indexed. Superseded content is a source problem, so solve it at the source.

> A company is building a rag assistant over its internal documents. The source folders also contain adverts, general news, and articles about competitor companies.
>
> The assistant should answer only about the company's own products and processes.

Which approach is NOT advisable?

A Remove documents about other companies before indexing

B Index everything, so the retriever has the most context to choose from

C Tag documents with metadata such as product, and filter at query time

D Remove duplicate and boilerplate content before chunking

◆ PAUSE AND DECIDE

Which approach is NOT advisable?

A Remove other companies' content

Advisable. Nobody should retrieve it.

B Index everything

Not advisable. Noise competes with, and outranks, the right chunks.

C Metadata tags and filters

Advisable. Targets retrieval to what each query needs.

D Remove duplicates and boilerplate

Advisable. Frees places in the top results.

THE SIGNAL

In a NOT question, three options are good practice. The odd one out is usually the one that sounds generous: more data, more context.

What to carry forward

- Match each kind of knowledge to its home: **index, keyed table, tool, SQL, or not yet written**.
- A user-supplied ID means a **keyed lookup**, often Feature Serving, not another document.
- Good sources are authoritative, current, complete and permitted. Not simply more.
- Remove what nobody should retrieve. Tag and filter what only some queries need.
- Telling the model to ignore noise is the weakest fix.

> ON THE EXAM
>
> Next: module 07 gets the text out of PDFs, web pages and scans, with the right package for each.